Fault-Tolerant Distributed Systems – Lab record

Lab 1 – Group G: member 1, member 2, member 3, member 4

1. Claim

Under assumptions 3 servers, crash-stop failures, messages delayed, reordered or lost by partitions (never duplicated or corrupted), asynchronous timing, if at most $f = 2$ servers other than n0 fail by crashing or being partitioned from n0, the service guarantees linearizable read/write/cas, and answers to the clients of n0 and of every server still connected to n0. Linearizability holds for any number of failures, n0 included; but with n0 down no client is answered, and in a partition the clients of servers cut off from n0 are not answered.
Evidence: 19 Maelstrom lin-kv runs of d.py, all valid (table below), including 3 partition runs and 3 runs with n0 crashed; plus the structural argument: one authoritative copy, executed one operation at a time, the only one ever read.
Not covered: any availability when n0 is down (no failover: 0 crashes of n0 tolerated); n0 restarting without memory; duplicated messages; durability (backups' copies can be stale and divergent).

2. Design and predictions

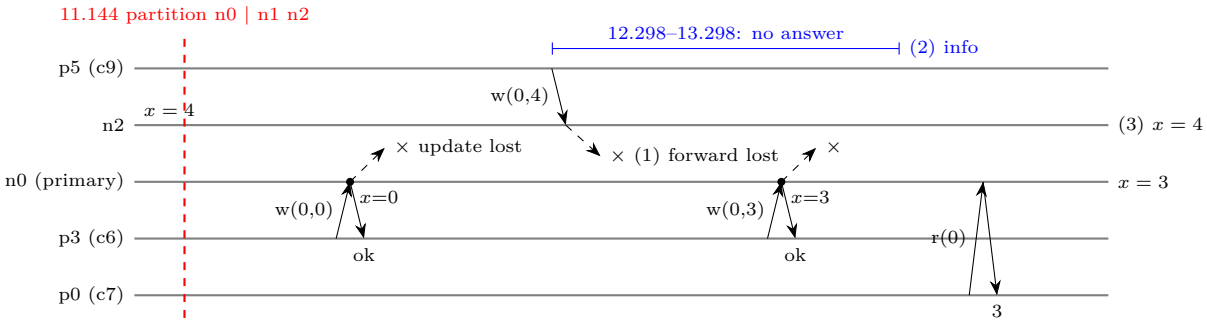
Strategy D, primary for everything. A backup (n1, n2) forwards every client request unchanged to n0 and relays n0's reply to the client; no timeout, no local answers. n0 executes the request on its copy; after a successful write/cas it sends update{key, value} (the new value) to the backups, fire and forget, then replies. Backups overwrite their copy on update; nobody reads it. No acks, sequence numbers, retries or failover (not part of D).

Scenario	Predicted	Observed: all valid; ok/fail/info	Why it differs
No faults	valid; all complete	150/59/0, 160/50/0, 125/74/0	–
--latency 50 --rate 50	valid; slower	744/270/0, 698/270/0, 711/295/0	median 108 ms at backups, 8 ms at n0: only 2 delayed hops, not 4
MS_CRASH=n0@5	valid; none after 5 s	0 ok after 5 s; 45/45/44 info	–
MS_CRASH=n1@5	valid; n1's clients stop	n1's clients: 0 ok after 5 s; 9/6/8 info	–
--nemesis partition	valid; cut-off side stops	124/55/8, 117/72/10, 143/53/4	–
--latency 100 --rate 100	valid	1039/361/0, 1017/378/0, 1026/356/0	extra: try to break it
MS_CRASH=n0@5-12	valid; resumes at 12 s	95/55/21 (1 run); 43 ok after 13 s	extra: n0 paused

(fail includes cas mismatches, missing keys, and reads that timed out: Maelstrom records a timed-out read as fail, not info.)

3. Evidence

Run runs/p4-partition (= store/lin-kv/20261016T180341.995+0100; partition, valid, 124/55/8). At 11.144 s the nemesis isolates n0 from n1 and n2 until the end. From then on, n0's clients complete 49 operations, n1's and n2's none: 0 forwarded requests reach n0 and 0 updates reach the backups (last apply: 10.484 s, $x = 4$).



(1) The forwarded write(0,4) is dropped by the partition; under D, n2 may not answer by itself. (2) p5 records info: the traces show the write was never executed, but p5 cannot tell this from "executed, reply lost"; the checker may leave it out, which is consistent with p0 reading 3. (3) The copies diverge (n2: 4, n0: 3), invisibly, since nobody reads n2. This does not violate our specification (linearizability, as the checker): every ok result comes from n0's one copy in n0's execution order. It does violate availability for 2/3 of the clients.

4. Limitations and open questions

- Valid runs are not a proof; our confidence rests on the argument that faults can only remove answers. 20 s runs, 3 nodes, few keys.
- No failover: as soon as a backup answers or takes over, the divergence of (3) becomes a stale read, and n0 may still be alive (split brain). Next: let n1 take over after a timeout, and observe the anomaly.
- Not injectable with ms.py: n0 restarting without its memory, duplicated messages. Next: longer partitions, more nodes, longer runs.
- Open: how to make a backup a safe successor — acks from (a majority of) backups before replying, ordered updates, agreement on who is primary.

5. AI log

- Claude Code was asked to "complete the lab for strategy D" (an answer directly request, against the guide's think-first workflow: noted here). It restated D and the open details first; the group checked them against its own design.
- It wrote kv.py, d.py and scripts to run each scenario 3 times and record verdict/counts; it kept to D (no acks, timeouts or failover) as AGENTS.md requires.
- Corrected/learnt from the runs, not from the assistant's prediction: timed-out reads are fail, not info; --latency delays only server-to-server messages; a backup timeout must not answer error 11 (n0 may have executed the request).
- (Group: add what you accepted, changed or rejected.)

6. Contributions

- member 1: …
- member 2: …
- member 3: …
- member 4: …